Séquences (PGSQL)

1. Définition

Une séquence est un Objet de base de base de données qui génère une suite d'entiers uniques, souvent pour des clés primaires.

Une séquence permet de Garantir l'unicité des identifiants, même dans un environnement concurrentiel

2. Création d'une séquence

Utilisez la commande CREATE SEQUENCE pour créer une séquence.

```
CREATE SEQUENCE nom_de_sequence
  [ AS type_de_donnees ]
  [ INCREMENT BY increment ]
  [ MINVALUE minvalue | NO MINVALUE ]
  [ MAXVALUE maxvalue | NO MAXVALUE ]
  [ START WITH start ]
  [ CACHE cache_size ]
  [ CYCLE | NO CYCLE ];
```

Explication des paramètres

- AS type_de_donnees : Définit le type de données de la séquence (SMALLINT, INTEGER ou BIGINT). La valeur par défaut est BIGINT.
- INCREMENT BY : La valeur d'incrémentation de la séquence (par défaut 1).
- START WITH : Le point de départ de la séquence (par défaut MINVALUE pour les séquences ascendantes).
- MINVALUE et MAXVALUE : Les bornes inférieure et supérieure de la séquence.
- CACHE : Le nombre de valeurs que le serveur pré-alloue en mémoire pour des performances améliorées.
- CYCLE ou NO CYCLE : Indique si la séquence doit recommencer lorsqu'elle atteint sa limite (MAXVALUE ou MINVALUE).

Exemple
Pour créer une séquence clients_id_seq qui commence à 100 et s'incrémente de 1 :

```
CREATE SEQUENCE idseq
  START WITH 100
  INCREMENT BY 1;
```

3. **Utilisation d'une séquence**

Les séquences sont utilisées via des fonctions spéciales.

- nextval('nom_de_sequence') : Récupère la valeur suivante de la séquence et la fait avancer. C'est la fonction la plus courante pour générer des identifiants uniques.
- currval('nom_de_sequence') : Renvoie la valeur la plus récemment obtenue pour la séquence dans la session actuelle. Attention, vous devez avoir déjà appelé nextval() dans cette session.

Exemple d'utilisation

```
-- Créer une table
CREATE TABLE clients (
   id INTEGER PRIMARY KEY,
   nom VARCHAR(100)
);
-- Insérer une nouvelle ligne en utilisant la séquence
INSERT INTO clients (id, nom)
VALUES (nextval('idseq'), 'Ali');
-- Utiliser la séquence comme valeur par défaut
-- Il est plus courant de lier la séquence à une colonne avec `DEFAULT`
CREATE TABLE produit (
   id INTEGER PRIMARY KEY DEFAULT nextval('ideq'),
   nom VARCHAR(100)
);
CREATE SEQUENCE id_seq OWNED BY produit.id;


-- Vous pouvez insérer des données sans spécifier la colonne 'produit_id'
INSERT INTO produits (nom) VALUES ('Ordinateur portable');
```

4. Modification d'une séquence

La commande ALTER SEQUENCE permet de modifier les paramètres d'une séquence existante.

```
ALTER SEQUENCE nom_de_sequence
  [ RESTART [ WITH valeur ] ]
  [ INCREMENT [ BY ] increment ]
  [ MAXVALUE maxvalue | NO MAXVALUE ]
  ...;
```

Exemple
Pour redémarrer une séquence à une nouvelle valeur :

```
ALTER SEQUENCE clients_id_seq RESTART WITH 1;
```

Suppression d'une séquence

La commande DROP SEQUENCE supprime un générateur de séquence.

```
DROP SEQUENCE [ IF EXISTS ] nom_de_sequence [, ...] [ CASCADE | RESTRICT ];
```

Explication des paramètres

- IF EXISTS : Supprime la séquence seulement si elle existe, évitant une erreur si elle est absente.
- CASCADE : Supprime automatiquement les objets qui dépendent de cette séquence (par exemple, si elle est liée à une colonne).
- RESTRICT : Empêche la suppression si d'autres objets en dépendent (comportement par défaut).

Exemple

```
DROP SEQUENCE IF EXISTS id_seq;
```

5. Le type SERIAL

PostgreSQL propose un raccourci pour la création de séquences : le pseudo-type SERIAL (ou BIGSERIAL). Quand vous définissez une colonne comme SERIAL, PostgreSQL crée automatiquement une séquence et l'associe à cette colonne en tant que valeur par défaut.

```
CREATE TABLE utilisateurs (
   id SERIAL PRIMARY KEY,
   nom VARCHAR(100)
);
-- PostgreSQL crée une séquence "utilisateurs_id_seq" en arrière-plan.

INSERT INTO utilisateurs (nom) VALUES ('Jean');
-- La colonne 'id' sera automatiquement remplie par la séquence.
```